



POLITECNICO
MILANO 1863

Software Engineering for Automation
A.Y. 2025–2026

Requirements Analysis and Specification Document

**Search-Based Test Generation for Autonomous UAV Obstacle
Avoidance Using a $(1 + 1)$ -Evolution Strategy**

Authors: Roberto Capone
Guglielmo Cattaneo

Instructor: Prof. Matteo Camilli

Version: 1.0

Release date: 17/07/2026

Revision History

Date	Version	Authors	Description
17/07/2026	1.0	Roberto Capone; Guglielmo Cattaneo	Initial complete release aligned with the final implementation contained in the project repository.

Contents

Revision History	1
1 Introduction	3
1.1 Purpose	3
1.2 Scope	3
1.3 Definitions, Acronyms, and Abbreviations	4
1.4 Reference Documents and Sources	5
1.5 Document Structure	5
2 Overall Description	6
2.1 Product Perspective and System Boundary	6
2.2 Stakeholders and User Characteristics	7
2.3 Operating Environment	7
2.4 Assumptions and Dependencies	7
2.4.1 Assumptions	7
2.4.2 External Dependencies	8
2.5 Competition and Domain Constraints	8
2.6 Product Functions	8
2.7 Principal Scenarios	9
2.7.1 SC-01: Successful Evolutionary Campaign	9
2.7.2 SC-02: Simulator Failure and Automatic Recovery	9
2.7.3 SC-03: Nominal Route Is Missing	9
2.7.4 SC-04: Critical Candidate Requires Confirmation	10
2.7.5 SC-05: Redundant Failure	10
2.7.6 SC-06: Random Baseline Campaign	10
2.8 Domain Model	11
2.8.1 Domain Entities	11
2.9 Functional Requirements	12
2.9.1 Invocation and Configuration	12
2.9.2 Mission and Route Handling	12
2.9.3 Candidate Generation and Validity	13
2.9.4 Simulation and Measurement	13
2.9.5 Evolutionary Search Control	14
2.9.6 Failure Archive and Delivery	14
2.9.7 Checkpointing, Recovery, and Logging	15
2.9.8 Visualisation and Inspection	16
2.10 Data Requirements	16
2.10.1 Input Data	16
2.10.2 Persistent Output Data	16
2.10.3 Checkpoint Integrity	17
2.11 Non-Functional Requirements	17

2.12 Excluded Requirements	18
3 Additional Models	19
3.1 Use-Case Model	19
3.1.1 UC-01: Run Generation Campaign	20
3.1.2 UC-02: Resume Interrupted Campaign	20
3.1.3 UC-03: Construct Final Delivery	20
3.2 Requirements-Level Sequence Model	21
3.3 Campaign State Model	22
3.4 Acceptance Criteria	23
3.5 Scenario-to-Requirement Traceability	23
4 References	24
A Requirement Index	25

Introduction

1.1 Purpose

This document specifies the requirements of the UAV Test Generator, a search-based software testing tool developed for the UAV Testing Competition. The system generates challenging obstacle configurations for the autonomous obstacle-avoidance behaviour of PX4-Avoidance, executes the resulting tests through the Aerialist test bench, and returns an ordered suite of failing test cases.

The document has four objectives:

1. define the system boundary and the relevant external actors;
2. state functional and non-functional requirements in a verifiable form;
3. describe the domain, principal scenarios, assumptions, dependencies, and constraints;
4. provide requirements-level behavioural models to support the subsequent design and implementation documents.

The intended audience consists of the project authors, the course instructor, maintainers of the generator, competition evaluators, and readers who need to verify consistency between requirements, design, implementation, and experimental results.

1.2 Scope

The UAV Test Generator is a command-line application that receives:

- an Aerialist YAML case-study description containing the UAV configuration, mission plan, and simulation environment; and
- a positive integer defining the maximum number of simulation attempts available to the generator.

The system produces one or more test cases containing between one and three non-overlapping box-shaped obstacles. Candidate configurations are evaluated in simulation. A test is considered failing when the minimum distance between the UAV trajectory and at least one obstacle is below the safety threshold of 1.5 m. The search objective is the continuous minimum distance, whereas the final delivery order additionally accounts for failure severity, number of obstacles, and execution time.

The implemented search method is a $(1 + 1)$ -Evolution Strategy with adaptive mutation, restart, targeted obstacle insertion, persistent checkpointing, and failure diversity management. An independent random-search baseline is included for controlled experimental comparison.

The following elements are outside the system boundary and are not modified by this project:

- PX4-Avoidance and the PX4 autopilot software;

- the Gazebo simulator and its physical models;
- the Aerialist execution framework;
- the official mission definitions and competition evaluator;
- the host operating system, Docker engine, and container runtime.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
Aerialist	Simulation-based UAV test bench that configures and executes PX4/Gazebo tests and returns flight logs.
Archive	Bounded in-memory collection of selected failing individuals used to construct the final delivery suite.
Candidate / Individual	One obstacle configuration together with its measured fitness, trajectory, completion flag, ranking data, and persistent artifact reference.
Case study	Input YAML description of a UAV mission and simulation environment without generated obstacles.
Checkpoint	Persistent JSON representation of the search state, sufficient to continue a run after process or simulator failure.
DTW	Dynamic Time Warping; a trajectory-alignment distance used by the implementation to identify similar failure behaviours.
ES	Evolution Strategy. The implemented search is a $(1 + 1)$ -ES with one current parent and one generated child per iteration.
Failing test	A test for which the observed minimum UAV–obstacle distance is lower than 1.5 m.
Fitness	Continuous search objective equal to the minimum UAV–obstacle distance observed for a candidate; lower values are better.
Genome	Ordered list of one to three obstacle genes. Each gene is $[l, w, h, x, y, r]$.
Hard fail	Collision or near-collision, operationally associated with a minimum distance close to zero.
IoU	Intersection over Union of two obstacle footprints. Used as a geometric fallback when trajectory data are unavailable and for restart novelty.
Operator	Human user who starts, monitors, or stops a generation campaign.
PX4-Avoidance	Vision-based autonomous obstacle-avoidance system under test.
SITL	Software-in-the-loop simulation.
Soft fail	Safety-distance violation with minimum distance below 1.5 m, whether or not the mission is completed.
Test budget	Maximum number of simulation attempts that may be consumed by one generator run.
Test suite	Ordered set of generated test cases exported for evaluation.
ULog	PX4 flight-log format containing the recorded trajectory and telemetry.

Term	Definition
YAML	Human-readable data-serialization format used by Aerialist test descriptions.

1.4 Reference Documents and Sources

The requirements are derived from the following authoritative sources:

1. the Software Engineering for Automation project guideline;
2. the UAV Testing Competition repository and case-study format;
3. the final implementation contained in the project repository;
4. the 2024 and 2025 UAV Testing Competition reports describing validity constraints, simulation budgets, failure scoring, and diversity concerns;
5. the Aerialist and PX4 references listed in Chapter 4.

1.5 Document Structure

Chapter 2 describes the product perspective, stakeholders, scenarios, domain model, product functions, constraints, and detailed requirements. Chapter 3 provides requirements-level use-case, sequence, and state models together with acceptance criteria and scenario traceability. Chapter 4 lists the references.

Overall Description

2.1 Product Perspective and System Boundary

The UAV Test Generator is an orchestration and search layer built above Aerialist. It does not implement flight control or physics. Instead, it constructs environment variants, delegates simulation to Aerialist, interprets the resulting trajectory, and manages the search and final test-suite selection.

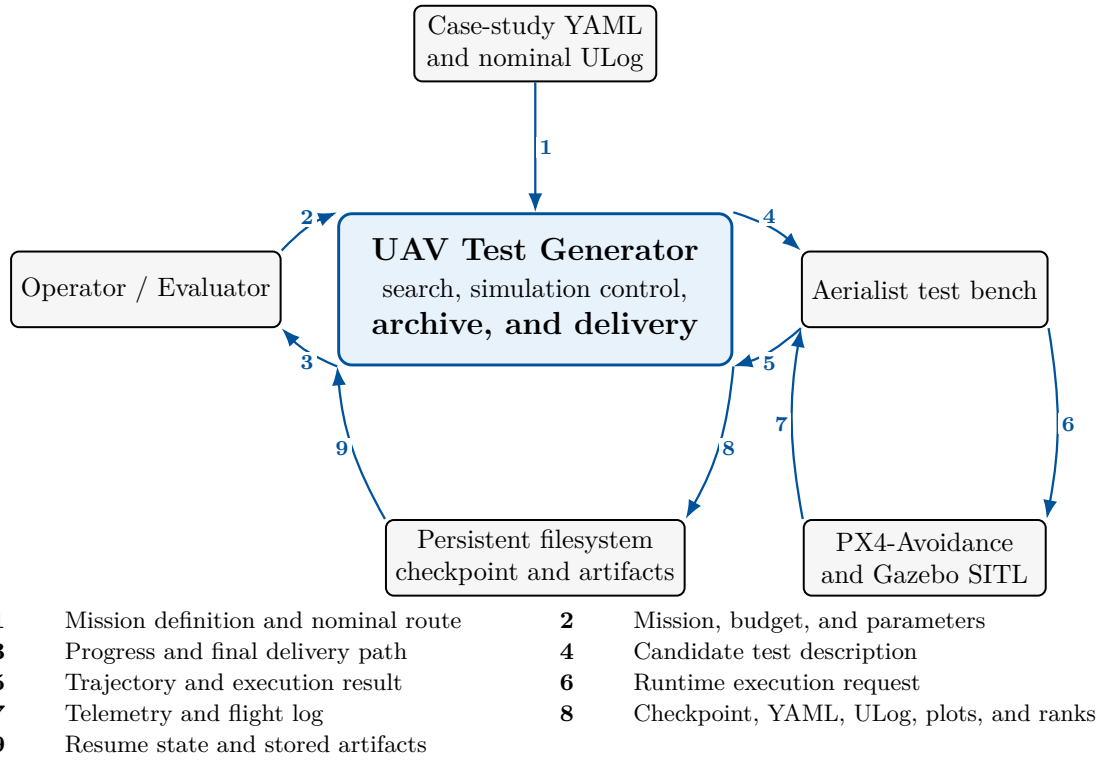


Figure 2.1: System context and externally observable information flows. The generator is treated as a black box at requirements level.

2.2 Stakeholders and User Characteristics

Stakeholder	Interest	Relevant characteristics
Project developer	Correctness, maintainability, testability, and extensibility of the generator.	Familiar with Python, Docker, search-based testing, and the repository structure.
Competition evaluator	Automated execution, valid outputs, bounded resource use, and ordered failing tests.	Interacts through the prescribed command-line interface and generated files.
Experiment operator	Reproducible campaign execution, clear logs, crash recovery, and result inspection.	Comfortable with Linux shell commands and environment variables.
Course instructor / reviewer	Consistency among requirements, design, code, tests, and documented results.	Requires precise models, traceability, and technically justified choices.
UAV software engineer	Actionable failing scenarios and interpretable flight artifacts.	Understands UAV missions and flight logs but need not understand the search implementation.

2.3 Operating Environment

The reference environment is:

- a Linux host with Docker support;
- the official Aerialist Docker image;
- Python 3 and the dependencies declared by the repository;
- a persistent host directory mounted into the container for code, checkpoints, logs, and generated tests;
- sufficient CPU, memory, and disk resources to execute PX4/Gazebo software-in-the-loop simulations.

The tool is command-line based and does not require a graphical interface. Simulations may run headlessly. A non-headless mode may be enabled for visual inspection when an X11 display is available.

2.4 Assumptions and Dependencies

2.4.1 Assumptions

A-1: The supplied case-study YAML is syntactically valid and can be parsed by Aerialist.

A-2: The mission, vehicle configuration, simulator, and required resources referenced by the case study are available to Aerialist.

A-3: Aerialist returns at least one execution result containing a trajectory and a log file for successful simulations.

A-4: The minimum distance computed by Aerialist is an adequate observable for detecting safety-distance violations.

- A-5:** The persistent output directory remains available across container restarts.
- A-6:** The host clock and filesystem support unique run-directory creation and atomic file replacement.
- A-7:** Simulator non-determinism may cause repeated executions of the same test case to produce different trajectories and distances.

2.4.2 External Dependencies

- D-1:** Aerialist APIs for loading test descriptions, creating obstacles, executing tests, extracting trajectories, and serialising YAML.
- D-2:** PX4-Avoidance and Gazebo for software-in-the-loop execution.
- D-3:** NumPy for trajectory aggregation and numerical processing.
- D-4:** Matplotlib for plot generation.
- D-5:** PyYAML for reading obstacle definitions in plotting utilities.
- D-6:** Shapely geometry operations provided through Aerialist obstacle objects.
- D-7:** Docker for the reference deployment and crash-restart wrapper.

2.5 Competition and Domain Constraints

The following constraints are mandatory for generated test cases:

- C-1:** A test case shall contain at least one and at most three box-shaped obstacles.
- C-2:** Every obstacle shall be placed on the ground, with $z = 0$.
- C-3:** Obstacle dimensions and poses shall remain within the official ranges encoded by the project configuration:

$$2 \leq l \leq 20, \quad 2 \leq w \leq 20, \quad 10 \leq h \leq 25, \\ -40 \leq x \leq 30, \quad 10 \leq y \leq 40, \quad 0^\circ \leq r \leq 90^\circ.$$

- C-4:** The complete rotated footprint of every obstacle shall fit inside the legal placement area; validity shall not be checked only at the obstacle centre.
- C-5:** Obstacles in the same test case shall not overlap in the horizontal plane.
- C-6:** The generator shall not modify the mission plan, UAV software configuration, or non-obstacle simulation properties of the input case study.
- C-7:** The number of simulation attempts shall not exceed the declared budget. Retries, route bootstrapping, and confirmation runs consume budget.
- C-8:** A single simulation shall be subject to a configurable timeout, whose reference value is 500 s.
- C-9:** The final delivery shall contain no more than the configured archive capacity, whose reference value is 20 tests.

2.6 Product Functions

At a high level, the product shall:

1. parse the command-line invocation and configure the selected generator;
2. load the mission and obtain a nominal route if adaptive sampling is enabled;

3. initialise or restore the search state;
4. generate valid obstacle configurations;
5. execute simulations while respecting budget and timeout constraints;
6. compute failure severity, completion information, trajectory descriptors, and ranking data;
7. evolve candidates through selection, mutation, step-size adaptation, and restart;
8. persist all discovered failures and the resumable search state;
9. construct a diverse, ordered final delivery suite;
10. provide a random-search baseline with the same evaluation and output infrastructure.

2.7 Principal Scenarios

2.7.1 SC-01: Successful Evolutionary Campaign

Initial condition: no checkpoint exists, the mission is valid, and the simulator is operational.

Flow:

1. The operator invokes the generator with a mission and a budget.
2. The system loads the mission and nominal route, or performs a route bootstrap when required.
3. The system evaluates an initial set of candidates and selects the best as the parent.
4. Until the budget is exhausted, the system mutates the parent, executes the child, updates the parent and mutation scale, archives failures, and checkpoints the state.
5. The system selects trajectory-distinct failures, orders them by delivery score, copies their artifacts into the delivery directory, and reports its location.

2.7.2 SC-02: Simulator Failure and Automatic Recovery

Initial condition: a candidate is marked as pending in the checkpoint and the simulator or container terminates unexpectedly.

Flow:

1. The wrapper detects a non-zero container exit code.
2. The wrapper starts a new container using the same persistent output directory.
3. The generator loads the checkpoint, archive, parent, mutation scale, route, and pending genes.
4. The interrupted candidate is re-executed from the beginning.
5. Search continues until the global budget or restart limit is reached.

2.7.3 SC-03: Nominal Route Is Missing

Initial condition: adaptive sampling is enabled, but no usable nominal ULog exists.

Flow:

1. The system runs one simulation without generated obstacles.
2. The resulting trajectory becomes the nominal route and its last point becomes the destination.
3. The system tunes positional mutation scales to the route corridor.

4. The route and destination are persisted in the checkpoint so that bootstrapping is not repeated after a crash.

2.7.4 SC-04: Critical Candidate Requires Confirmation

Initial condition: a candidate produces an observed minimum distance below the configured confirmation threshold.

Flow:

1. The system schedules the configured number of additional executions, provided budget remains.
2. It aggregates the execution results, retains the most severe observed test artifact, computes the average competition point and execution time, and averages the resampled trajectories.
3. The aggregate candidate is used by the archive and delivery logic.

2.7.5 SC-05: Redundant Failure

Initial condition: a new failure has a trajectory within the configured DTW threshold of an archived failure.

Flow:

1. The system permanently saves the new failure in the complete failure-history directory.
2. If the new failure is more severe, it replaces the archived representative of the trajectory basin.
3. Otherwise it is excluded from the delivery archive and contributes additional stagnation pressure.

2.7.6 SC-06: Random Baseline Campaign

Initial condition: the generator mode is configured as random.

Flow:

1. The system uses the same case loading, legal domain, simulation execution, budget accounting, failure archive, checkpoint, and delivery logic as the evolutionary generator.
2. Every new candidate is sampled independently and does not depend on the parent, mutation scale, or targeted restart.
3. The generated results can be compared with the evolutionary strategy without changing the evaluation infrastructure.

2.8 Domain Model

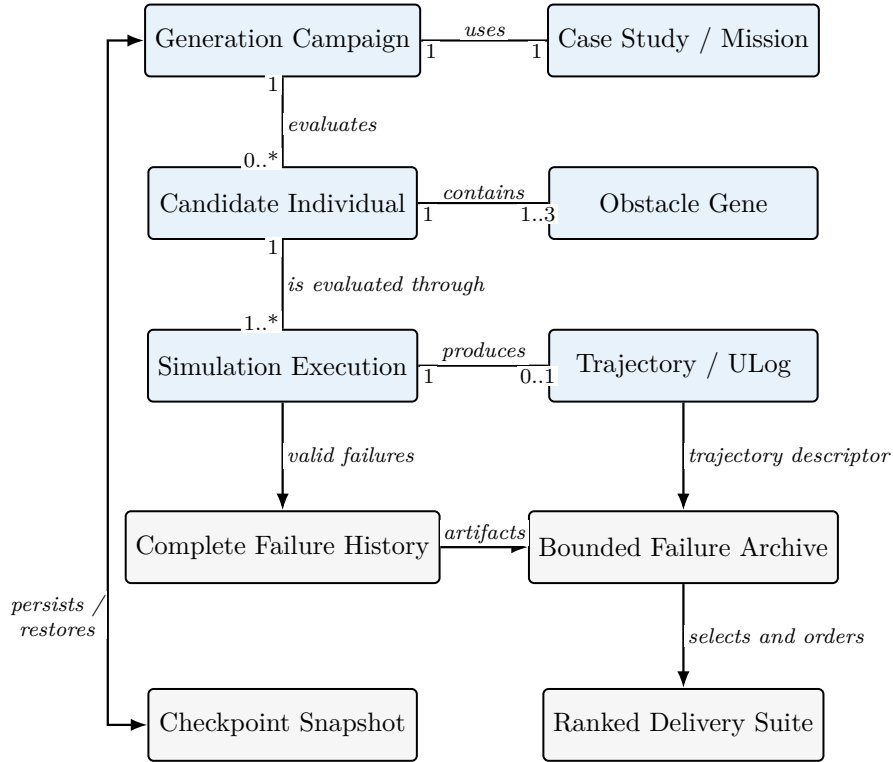


Figure 2.2: Requirements-level domain model, separating durable evidence, bounded archive state, and final delivery.

2.8.1 Domain Entities

Entity	Meaning and relevant information
Generation Campaign	One invocation on one case study with one global budget, configuration, output directory, and selected generation method.
Case Study / Mission	Immutable Aerialist YAML input defining the UAV, mission, simulator, and environment to which generated obstacles are added.
Candidate Individual	Candidate scenario represented by genes and enriched after execution with fitness, minimum distance, completion state, average point, duration, trajectory, and file references.
Obstacle Gene	Six-dimensional tuple $[l, w, h, x, y, r]$ defining one box obstacle; z is fixed to zero.
Simulation Execution	One budget-consuming attempt to run a candidate, possibly ending in success, timeout, execution failure, or process crash.
Trajectory / ULog	Recorded UAV motion used for distance measurement, mission-completion information, plotting, and behavioural diversity.
Failure Archive	Bounded set of representative failing individuals, one best representative per sufficiently distinct trajectory basin.
Delivery Suite	Ordered subset copied from persistent failures and intended for external evaluation.

Entity	Meaning and relevant information
Checkpoint	Atomic persistent snapshot of the search state and pending candidate.

2.9 Functional Requirements

Each requirement is assigned a unique identifier and a priority. Mandatory requirements define the accepted product. “Should” requirements are important quality-enhancing functions implemented by the reference system. “Could” requirements are optional operational conveniences.

2.9.1 Invocation and Configuration

ID	Priority	Requirement
FR-01	Mandatory	The system shall provide a command-line operation generate <code><case-study></code> <code><budget></code> .
FR-02	Mandatory	The system shall reject or terminate unsuccessfully when the case-study path cannot be loaded, the budget is not an integer, or an unrecoverable initialisation error occurs.
FR-03	Mandatory	The system shall support selecting the evolutionary generator or the random-search baseline without changing the command-line contract.
FR-04	Mandatory	The system shall provide documented default values for all parameters required by an evaluator that supplies no optional environment variables.
FR-05	Should	The system shall allow search, evaluation, timeout, archive, diversity, and reproducibility parameters to be overridden through environment variables.

2.9.2 Mission and Route Handling

ID	Priority	Requirement
FR-06	Mandatory	The system shall load the input case study through the Aerialist data model and preserve all input properties except the generated obstacle list.
FR-07	Should	When adaptive sampling is enabled, the system shall attempt to extract a nominal (x, y) route and destination from a corresponding ULog.
FR-08	Should	If no nominal route is available, the system shall attempt one or more budget-accounted obstacle-free bootstrap simulations and use the recorded trajectory as the route.
FR-09	Should	The system shall tune positional sampling or mutation scales to the extent of the obtained route while retaining the official legal bounds.

ID	Priority	Requirement
FR-10	Mandatory	Failure to obtain a nominal route shall not make the generator unusable; the system shall fall back to a documented fixed sampling region.

2.9.3 Candidate Generation and Validity

ID	Priority	Requirement
FR-11	Mandatory	The system shall represent a candidate with one to three obstacle genes $[l, w, h, x, y, r]$.
FR-12	Mandatory	Every generated obstacle shall satisfy the size, position, rotation, and ground-placement constraints defined in Section 2.5.
FR-13	Mandatory	The system shall account for obstacle rotation when ensuring that the complete obstacle footprint fits within the legal area.
FR-14	Mandatory	The system shall prevent or repair overlapping obstacles before simulation.
FR-15	Mandatory	In evolutionary mode, the system shall generate each ordinary child by mutating the current parent and may add or remove an obstacle while preserving the allowed cardinality.
FR-16	Should	When adding an obstacle to a candidate with a recorded trajectory, the system should be able to target an observed escape gap and orient the added obstacle across the local direction of motion.
FR-17	Mandatory	In random mode, each ordinary candidate shall be sampled independently of previous candidates, the current parent, and the mutation scale.

2.9.4 Simulation and Measurement

ID	Priority	Requirement
FR-18	Mandatory	The system shall execute candidates through Aerialist using a copy of the original case study enriched with the generated obstacles.
FR-19	Mandatory	Every simulation attempt, including retries, route bootstrap, and confirmation executions, shall increment the global budget counter exactly once.
FR-20	Mandatory	The system shall not start a simulation attempt after the global budget has been exhausted.
FR-21	Mandatory	The system shall enforce a configurable timeout for each simulation attempt and cancel the timeout after the attempt terminates.
FR-22	Should	The system shall retry failed or timed-out simulation attempts up to a configurable limit while budget remains.

ID	Priority	Requirement
FR-23	Mandatory	For every successful execution, the system shall compute the minimum distance between the trajectory and the generated obstacles.
FR-24	Mandatory	The candidate fitness shall be the minimum observed distance; a lower value shall represent a better search result.
FR-25	Should	The system shall determine whether the trajectory reaches a configurable neighbourhood of the nominal destination and record this information without using it to suppress a genuine safety violation.
FR-26	Should	The system shall calculate the execution duration, competition point band, and a fixed-length resampled (x, y) trajectory when sufficient log data are available.
FR-27	Should	When the observed distance is below a configured criticality threshold, the system shall schedule additional confirmation executions, subject to the remaining budget, and aggregate the observations.

2.9.5 Evolutionary Search Control

ID	Priority	Requirement
FR-28	Mandatory	On a fresh evolutionary run, the system shall evaluate a configurable set of initial candidates and select the lowest-fitness candidate as the initial parent.
FR-29	Mandatory	After each child evaluation, the system shall replace the parent if and only if the child has strictly lower fitness.
FR-30	Mandatory	The system shall maintain an improvement history and adapt the mutation scale according to a documented success-rule policy within configured minimum and maximum limits.
FR-31	Mandatory	The system shall maintain a stagnation counter and initiate a restart after a configured number of non-improving generations.
FR-32	Should	A diversified restart should prefer a mission region insufficiently represented by the failure archive and should select among multiple candidates using geometric novelty.
FR-33	Should	When the spatial regions are already covered, the system should probabilistically intensify the search around the most severe archived failure.
FR-34	Should	A redundant failure should increase stagnation pressure so that the search leaves a behaviourally exhausted region earlier.

2.9.6 Failure Archive and Delivery

ID	Priority	Requirement
FR-35	Mandatory	A candidate with minimum distance below 1.5 m shall be treated as a valid failure independently of whether the destination was reached.
FR-36	Mandatory	Every valid failure with available execution artifacts shall be persisted immediately in a complete failure-history directory before the next candidate is evaluated.
FR-37	Mandatory	The persisted failure artifacts shall include the generated YAML and, when available, the corresponding ULog and official-style plot.
FR-38	Mandatory	The system shall maintain a bounded delivery archive and shall identify behaviourally redundant failures using trajectory distance when trajectory data are available.
FR-39	Mandatory	If a new failure belongs to the same trajectory basin as an archived failure, the system shall retain the one with the lower minimum distance as the archive representative.
FR-40	Mandatory	When the archive is full, a new distinct failure shall enter only if it improves the archive according to the documented delivery-ranking criterion.
FR-41	Mandatory	The final delivery shall contain an ordered subset whose retained trajectories satisfy the configured separation rule whenever trajectory data are available.
FR-42	Mandatory	The delivery order shall be descending in the score
$S(t) = \frac{10 \bar{p}(t)}{n_{\text{obs}}(t)^2 \bar{T}(t)},$ where $\bar{p}$ is the average point, n_{obs} is the number of obstacles, and $\bar{T}$ is the average execution time in minutes.		
FR-43	Mandatory	The final directory shall use stable rank-prefixed basenames such that rank01 denotes the highest-ranked delivered test.
FR-44	Mandatory	Rebuilding the final selection shall be idempotent and shall not delete the complete failure history.

2.9.7 Checkpointing, Recovery, and Logging

ID	Priority	Requirement
FR-45	Mandatory	The system shall save a checkpoint before executing a candidate and after completing each initial seed or generation.
FR-46	Mandatory	Checkpoint creation shall use an atomic replacement strategy so that interruption during writing does not replace the last valid checkpoint with a partial file.
FR-47	Mandatory	A checkpoint shall contain sufficient serialisable data to restore the budget counter, search parameters, parent, archive, pending candidate, initialisation state, nominal route, and destination.

ID	Priority	Requirement
FR-48	Mandatory	On restart, the system shall restore a valid checkpoint from the same output directory and continue the campaign without resetting the global budget or archive.
FR-49	Mandatory	A candidate marked as pending at the time of failure shall be executed again from the beginning using the same genes.
FR-50	Should	The reference launcher should restart a failed container automatically up to a configurable limit and preserve host ownership of generated artifacts.
FR-51	Mandatory	The system shall record detailed diagnostics in a persistent debug log and concise progress information on standard output.
FR-52	Mandatory	At successful termination, the system shall report the final delivery-directory location and the number of delivered tests.

2.9.8 Visualisation and Inspection

ID	Priority	Requirement
FR-53	Should	For each persisted failure with a readable ULog, the system should generate a plot containing x , y , z , and yaw versus time and a top-down trajectory view with rotated obstacle footprints.
FR-54	Should	Plot-generation failure shall be logged but shall not terminate the search or invalidate otherwise usable YAML and ULog artifacts.
FR-55	Could	The launcher may support a non-headless simulation mode for interactive debugging while retaining headless execution as the default.

2.10 Data Requirements

2.10.1 Input Data

The minimum input is a case-study path and a simulation budget. Optional configuration is supplied through environment variables. The case study is interpreted by Aerialist and must contain the information necessary to instantiate the UAV, simulator, mission, and initial environment.

2.10.2 Persistent Output Data

A generation campaign shall produce a self-contained output tree containing, as applicable:

- a persistent debug log;
- an atomic JSON checkpoint;
- a complete failure-history directory containing YAML, ULog, and plot artifacts;
- a delivery directory containing ordered rank-prefixed copies;
- any launcher or simulator diagnostics required to analyse failures.

2.10.3 Checkpoint Integrity

Only JSON-safe values shall be persisted. External Aerialist objects shall not be serialised directly. Values representing unavailable infinity shall be encoded using an explicit nullable representation. Persistent references to failure artifacts shall be relative to the campaign output directory to preserve relocatability.

2.11 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Correctness	The generator shall enforce the obstacle cardinality, legal-domain, ground-placement, and non-overlap constraints before executing a candidate.
NFR-02	Budget safety	The recorded number of attempts shall never exceed the declared simulation budget.
NFR-03	Recoverability	After an unexpected termination, all failures persisted before the termination and the latest valid checkpoint shall remain usable.
NFR-04	Reliability	A timeout, failed simulation, missing optional plot dependency, or unreadable optional nominal log shall be handled without corrupting persistent state.
NFR-05	Reproducibility	Supplying the same explicit pseudo-random seed and configuration shall reproduce the generator’s sampling and mutation decisions, although simulator outputs may remain non-deterministic.
NFR-06	Performance	Trajectory comparison shall operate on a bounded resampled representation rather than unbounded raw logs, and archive size shall remain bounded.
NFR-07	Portability	The reference deployment shall run in Docker on a Linux host without requiring installation of PX4/Gazebo directly on the host.
NFR-08	Maintainability	Search configuration, geometry and scoring, data model, orchestration, execution wrapper, plotting, and CLI concerns shall be separated into identifiable modules.
NFR-09	Extensibility	A baseline generator shall reuse common execution and delivery infrastructure while replacing the candidate-generation policy through subclassing or an equivalent variation mechanism.
NFR-10	Observability	Logs shall expose mission, budget progress, generated obstacle parameters, measured distance, completion information, restart events, archive decisions, and final delivery information.
NFR-11	Usability	A standard campaign shall be startable through one documented shell command and shall print the output location at termination.
NFR-12	Traceability	Every mandatory functional requirement shall map to one or more design components and implementation modules in the Design Document.

ID	Category	Requirement
NFR-13	Data preservation	Constructing or reconstructing the delivery suite shall never remove the complete failure-history artifacts.
NFR-14	Graceful degradation	Missing trajectory, flight-duration, or optional plotting data shall trigger documented fallback behaviour rather than an unhandled exception.
NFR-15	Testability	Pure geometry, scoring, serialisation, archive, and selection behaviour shall be separable from expensive simulator execution so that automated tests can exercise deterministic logic.

2.12 Excluded Requirements

The following capabilities are explicitly outside the project scope:

- modification or verification of the internal PX4-Avoidance source code;
- formal proof of collision avoidance or simulator fidelity;
- real-world UAV flight execution;
- optimisation of wind, lighting, sensor noise, or non-box obstacle types;
- a graphical campaign-management application;
- distributed Kubernetes scheduling implemented by the generator itself;
- permanent deletion of generated failures or automatic upload to an external evaluator.

Additional Models

3.1 Use-Case Model

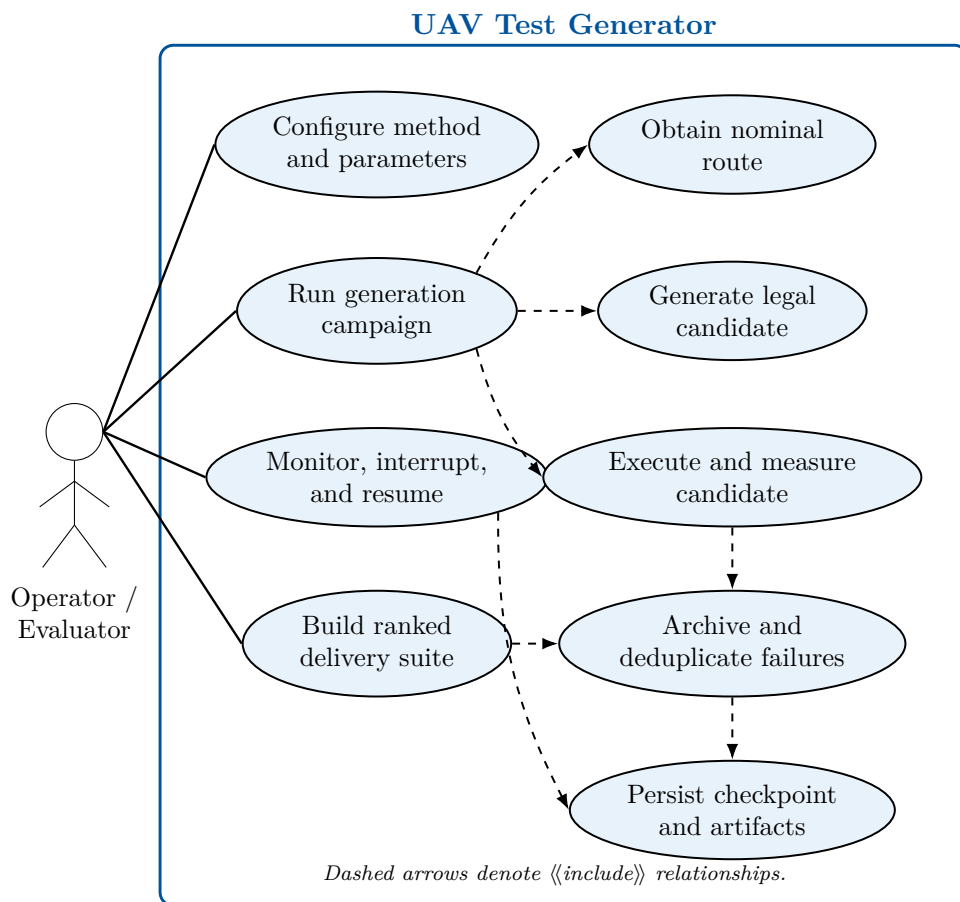


Figure 3.1: Primary evaluator-facing use cases and the campaign services they include.

3.1.1 UC-01: Run Generation Campaign

Primary actor	Operator or competition evaluator
Preconditions	Valid execution environment; readable case-study path; positive integer budget
Trigger	Invocation of the generate command
Main success result	Budget-bounded campaign completes and an ordered delivery directory is produced
Failure result	Process exits non-zero with persistent diagnostics; previously saved state remains available
Related requirements	FR-01–FR-05, FR-18–FR-24, FR-28–FR-52

3.1.2 UC-02: Resume Interrupted Campaign

Primary actor	Launcher / operator
Preconditions	Persistent output directory contains a valid checkpoint
Trigger	Process or container restarts after an unexpected termination
Main success result	Search state is restored and the pending candidate is re-executed without resetting the archive or budget
Alternative result	If the checkpoint is unreadable, the system reports the condition and performs the documented clean-start behaviour
Related requirements	FR-45–FR-51; NFR-03, NFR-04, NFR-13

3.1.3 UC-03: Construct Final Delivery

Primary actor	Competition evaluator
Preconditions	The campaign has an archive and/or persistent failure artifacts
Trigger	Search budget is exhausted or final selection is explicitly rebuilt
Main success result	A rank-ordered, diversity-filtered suite is copied into the delivery directory
Postconditions	Complete failure history is unchanged; obsolete rank copies may be removed only from the delivery directory
Related requirements	FR-35–FR-44, FR-52; NFR-13

3.2 Requirements-Level Sequence Model

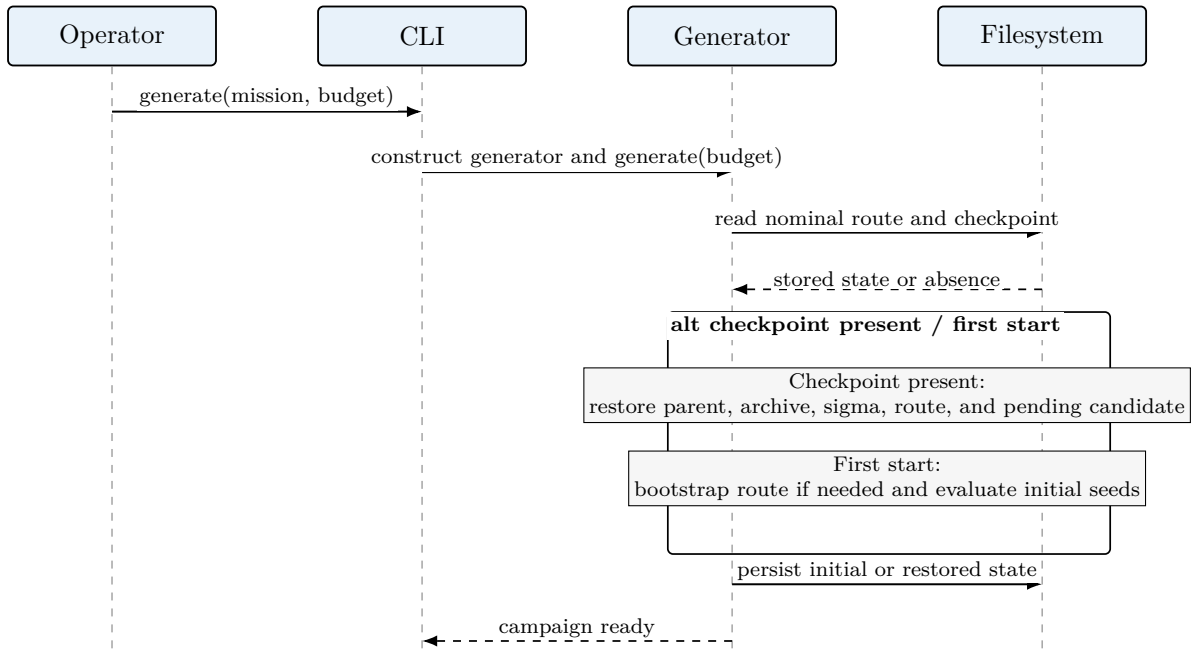


Figure 3.2: Requirements-level interaction for campaign invocation, initialisation, and recovery.

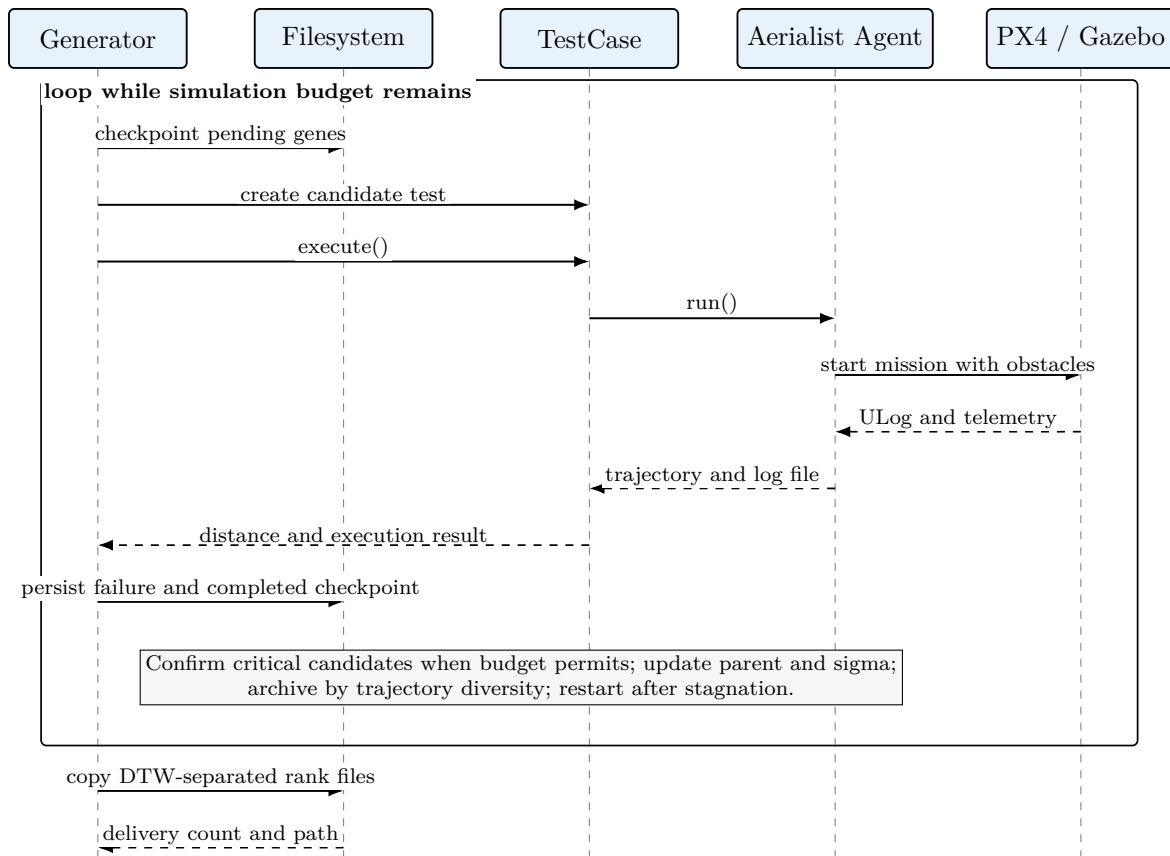


Figure 3.3: Requirements-level interaction for candidate execution, archive update, checkpointing, and final delivery.

3.3 Campaign State Model

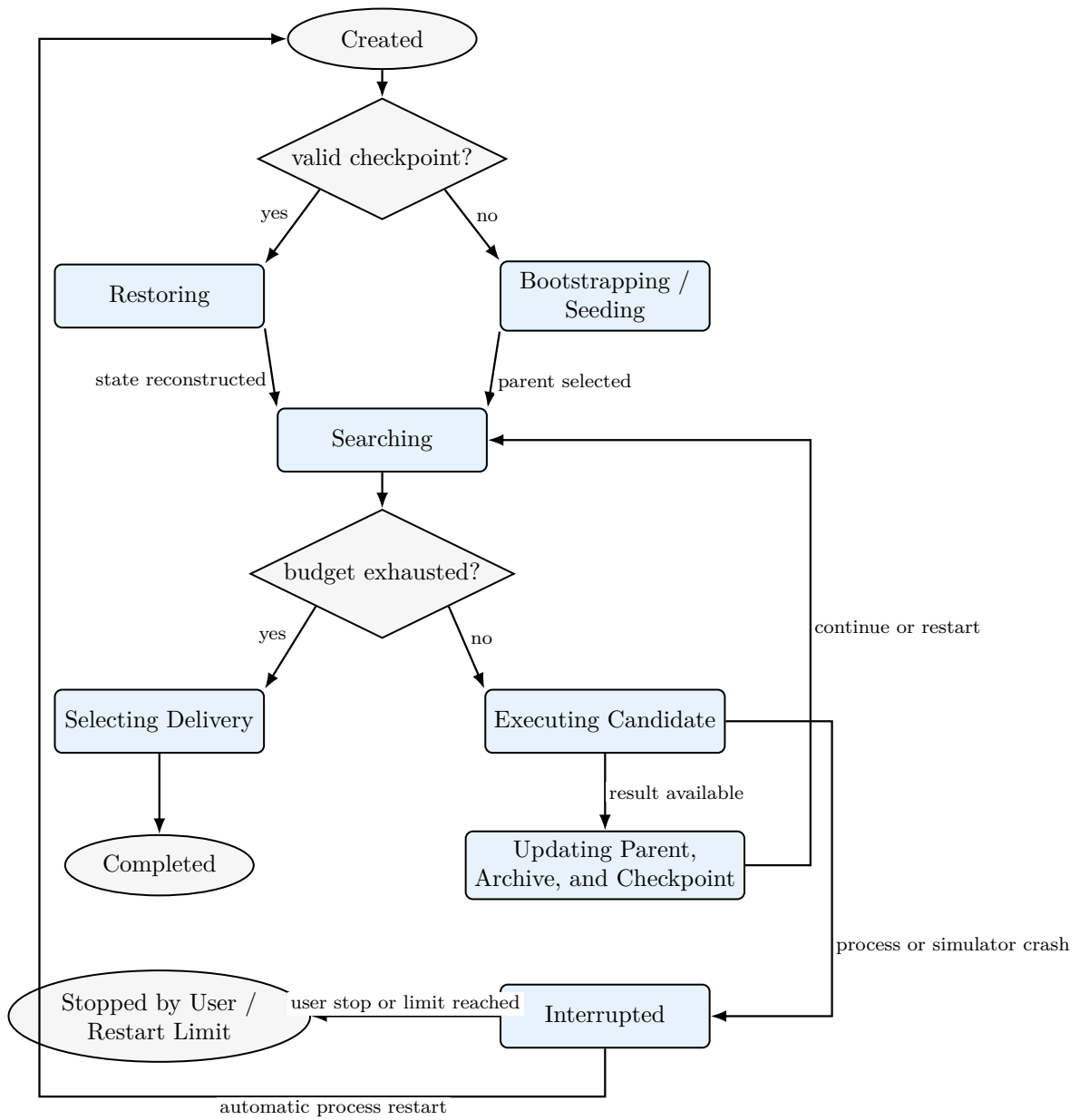


Figure 3.4: Campaign lifecycle and crash-recovery transitions.

Source state	Target state	Trigger
Searching	Executing candidate	A candidate is ready and budget remains.
Executing candidate	Searching	The attempt terminates and state updates complete.
Searching	Selecting delivery	The global budget is exhausted.
Executing candidate	Interrupted	The process or simulator terminates unexpectedly.
Interrupted	Created	The launcher permits another process restart; the new process then validates and restores the checkpoint.
Interrupted	Stopped / failed	The restart limit is reached or the operator stops the run.

3.4 Acceptance Criteria

The product is acceptable when all of the following conditions hold:

1. The documented command runs on the reference Docker environment and accepts a mission YAML and budget.
2. Automated tests confirm the legal obstacle domain, rotation-aware fitting, overlap rejection or repair, serialisation round-trip, scoring formulas, archive decisions, and final selection logic.
3. A campaign cannot consume more simulation attempts than its declared budget.
4. At least one deliberately interrupted campaign resumes from its checkpoint without losing already persisted failures.
5. Every delivered YAML contains one to three legal non-overlapping obstacles and is accompanied by available ULog and plot artifacts.
6. Delivery ranks are ordered by the documented score and rebuilding the delivery does not alter the complete failure-history directory.
7. The evolutionary and random generators can be invoked through the same external interface and generate separate campaign outputs.
8. The repository documentation identifies installation prerequisites, execution commands, parameter defaults, output structure, and authorship.

3.5 Scenario-to-Requirement Traceability

Scenario	Principal requirements
SC-01	FR-01, FR-06, FR-11–FR-24, FR-28–FR-46, FR-51–FR-54; NFR-01, NFR-02, NFR-08, NFR-11
SC-02	FR-19, FR-45–FR-51; NFR-03, NFR-04, NFR-13
SC-03	FR-07–FR-10, FR-19, FR-47–FR-49; NFR-04, NFR-14
SC-04	FR-19, FR-20, FR-23–FR-27, FR-35–FR-40; NFR-02, NFR-05
SC-05	FR-34–FR-41, FR-44; NFR-06, NFR-13
SC-06	FR-03, FR-17–FR-27, FR-35–FR-52; NFR-09

References

- [1] M. Camilli, *Software Engineering (for Automation) – Project Guideline*, Politecnico di Milano, A.Y. 2024–2025.
- [2] R. Capone and G. Cattaneo, *Search-Based Test Generation for Autonomous UAV Obstacle Avoidance Using a $(1+1)$ -Evolution Strategy*, Software Engineering Project Repository, 2026. Available: https://github.com/gulicatta/SE4A_RobertoCapone_GuglielmoCattaneo.
- [3] UAV Testing Competition, *Official Repository and Competition Documentation*. Available: <https://github.com/skhatiri/UAV-Testing-Competition>.
- [4] S. Khatiri et al., “SBFT Tool Competition 2024 – CPS-UAV Test Case Generation Track,” in *International Workshop on Search-Based and Fuzz Testing*, 2024.
- [5] S. Khatiri, T. Zohdinasab, P. Saurabh, D. Humeniuk, and S. Panichella, “ICST Tool Competition 2025 – UAV Testing Track,” in *IEEE International Conference on Software Testing, Verification and Validation*, 2025.
- [6] S. Khatiri, T. Zohdinasab, P. Saurabh, D. Humeniuk, and S. Panichella, “SBFT Tool Competition 2025 – UAV Testing Track,” in *International Workshop on Search-Based and Fuzz Testing*, 2025.
- [7] S. Khatiri, S. Panichella, and P. Tonella, “Simulation-based Testing of Unmanned Aerial Vehicles with Aerialist,” in *International Conference on Software Engineering*, 2024.
- [8] S. Khatiri, S. Panichella, and P. Tonella, “Simulation-based Test Case Generation for Unmanned Aerial Vehicles in the Neighborhood of Real Flights,” in *IEEE International Conference on Software Testing, Verification and Validation*, 2023.
- [9] L. Meier, D. Honegger, and M. Pollefeys, “PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms,” in *IEEE International Conference on Robotics and Automation*, 2015, pp. 6235–6240.

Requirement Index

Range	Subject
FR-01–FR-05	Invocation and configuration
FR-06–FR-10	Mission and route handling
FR-11–FR-17	Candidate generation and validity
FR-18–FR-27	Simulation and measurement
FR-28–FR-34	Evolutionary search control
FR-35–FR-44	Failure archive and delivery
FR-45–FR-52	Checkpointing, recovery, and logging
FR-53–FR-55	Visualisation and inspection
NFR-01–NFR-15	Quality attributes and cross-cutting constraints